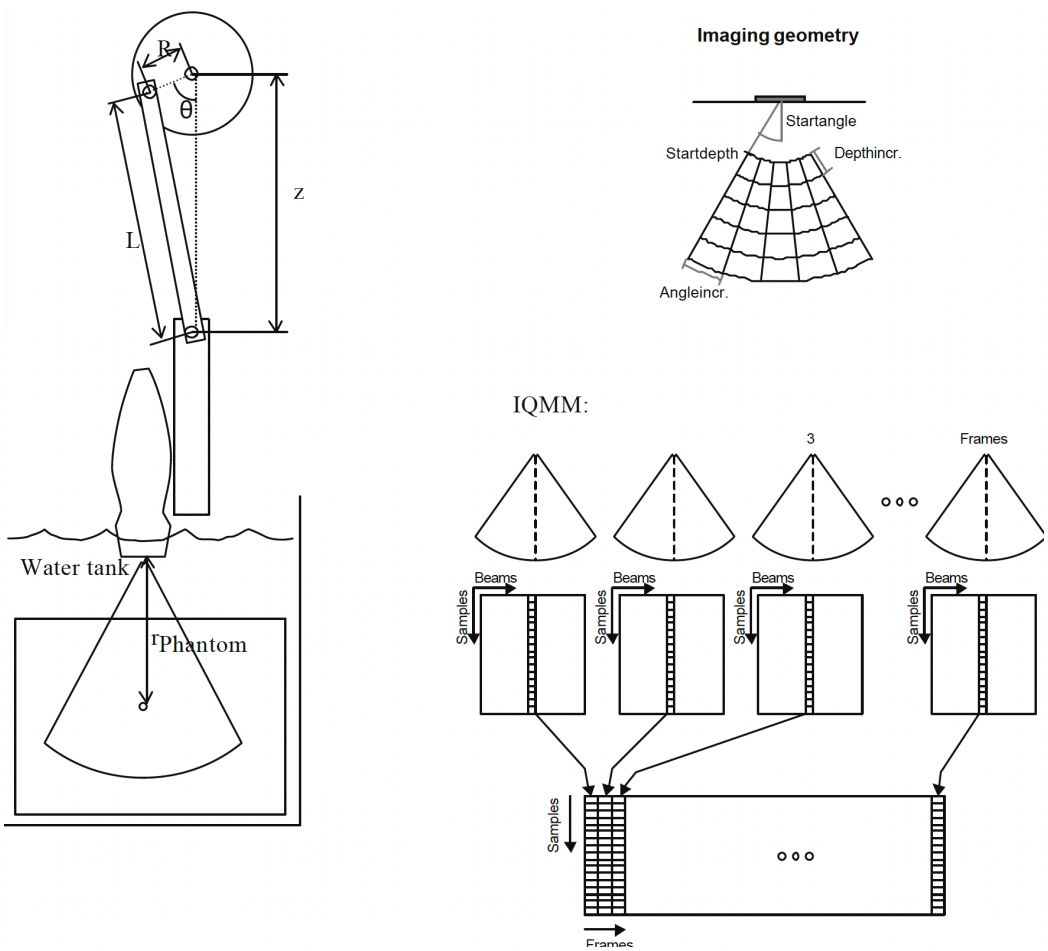
Exercise 8 – Spectral Doppler

In this exercise we will continue to work with ultrasound imaging of moving targets, and the estimation of velocities as used in blood flow imaging. Central to measuring velocity and displacement with ultrasound is the use of power spectral estimation. We will here use a lab dataset with motion to investigate different aspects of spectrum estimation, and in the end work on a real in vivo dataset to produce the final Doppler sonogram results.

The laboratory dataset was acquired in the ultrasound lab (many years ago) and is illustrated in the picture below. The ultrasound transducer is positioned to a rod and moved up and down in a cyclic manner (as seen left in the figure). A special recording setup images a stationary target in the water tank. The image geometry is depicted in the top right part of the figure. In this exercise, we will work with one-dimensional M-mode imaging and spectral-Doppler estimation, and to do this we will extract the center beam and plot this over time, as shown in the bottom right figure.



1. Load and plot the M-mode data (repetition from Ex. 7).

Download the datasets called *slowmotion_v2.mat*. You can read these datasets in Matlab directly or through the *scipy.io.loadmat* function in Python. The data contains both IQ-data and various parameters describing the geometry and setup. From Python these are available in the dictionary that is made by *loadmat*, where 'iq' is the IQ data, while 'pars' contains relevant parameters.

- The dimensions in the IQ data are: [range/depth sample, beam number, frame number].
- Relevant parameters included are:

  'fps': frame rate, the time axis can be made based on this

  'depth_start': The starting depth of the scan in [m]

  'depth_inc': The increment between depth samples in [m]

  'f0': The center frequency of the ultrasound pulse

- To read the parameters in Python, scipy.io.loadmat uses structured Numpy arrays, the parameters can be read as:

  data = scipy.io.loadmat(filename)

  pars = data['pars']

  fps = pars[0,0]['fps'].item()

  etc.

  (The item() is to flatten the returned arrays to scalars, needed to avoid Numpy warnings when using it.)

a) Extract the center beam of the IQ data for the two datasets and process this to an M-mode. Display it as an image in the log-domain using a greyscale colormap, with correct depth and time axis. Limit the dynamic range to 40dB and use depth and time axis calculated from the parameters.

   Tips:

   - Normalize and set vmin=-40, and vmax=0 in the imshow function in Python, or use the clim function in Matlab.
   - Create the depth and time axis using the parameters above (e.g. using the Numpy arange function in Python, or ':' operator in Matlab.

2. Spectral analysis – the Welch approach

We will now make an improved spectral spectrum estimator based on the Welch approach. This includes the following procedure:

- Sliding window *with overlap* to extract segments
- Smooth window function weighting of the data in each segment
- Periodogram estimator of the spectrum (abs(fft(seg)^2)
- Sliding window averaging of the spectrum estimates

You have already done most of this work in Exercise 8 but need to add an overlap option in the sliding window processing, and spectrum averaging. You can implement the procedure yourself as a programming exercise, but I have also supplied some code for getting started in the *exercise_9_tips.py / mat* file.

a) Repeat *Exercise 8-e* and generate the sonogram of the slowmotion data with the new Welch spectrum estimator. You can as a starting setup use a window length of 16, a 50% overlap and average 4 spectrum estimates. The window function can be Hamming. Compare to the result without overlap and averaging. What do you observe?
b) Change the overlap to 0.1 (90% overlap) and run it again. You can here potentially increase the number of averaged spectrums for even better results. What do you observe?
c) Play with the window length. 8, 16, 32, 64. Which one do you think looks best?

3. Clutter and clutter filtering

Let's look at the effect of clutter signal from surrounding tissue. Load the file called *slowmotion_clutter_v2.mat* (similar to the procedure for slowmotion_v2.mat).

a) Plot the M-mode as in 1). What do you see in the M-mode?

b) Generate and plot the resulting sonogram using the best settings you have found so far. What do you see?

c) What happens at zero frequency as you increase the window length? What does this tell us?

   Note: There is a tradeoff between the time resolution and frequency resolution. Using more samples provides better frequency resolution, but the number of unique

spectrums is lowered, hence poorer time resolution. By using overlap in the sliding window processing, we can find a better compromise.

d) Change the window function to be a boxcar instead of Hamming/Hanning and plot the spectrum again. What do you observe?

Note: The weighing function has a significant influence on the sidelobe level of the individual frequency component. Remember that the estimate of a single frequency component for a signal will be a sinc-function if a boxcar (no weighting function) is given. The sidelobes of the sinc is 13B below the main lobe. In comparison, the sidelobe level of a Hamming window is 42dB below the main lobe.

e) Let's design a suitable high-pass filter. Extract the power spectrum from one sonogram slice/time and plot it (Power vs. frequency).

What cut-off frequency is suitable to attenuate the clutter without influencing the blood signal?

What stop-band attenuation do you need to make sure that the clutter component is below the blood component?

Note: Designing a suitable high-pass filter is a trade-off between achieving a steep transition region from the pass band to stop band, and a sufficient stop band attenuation. In general, by increasing the FIR filter order, a better transition region and stop band attenuation is achieved. But another trade-off then emerges, as we also will need more samples to initialize the filter and limit the delay between input and output in a real-time system (which we need for ultrasound imaging). Here, sufficient stop band attenuation is achieved when the clutter signal power is well below that of the blood signal, and ideally below the noise level.

f) Design this FIR filter, for instance using the functions *scipy.signal.firwin* in Python or *fir1* in Matlab. Plot the filter frequency response on top of the sonogram slice in e).

Note: You can use the freqz function in Python/scipy or Matlab, or you can estimate it yourself. The filter frequency response for a FIR filter is simply the Fourier transform of the filter coefficients, which you can display as a power spectrum.

g) Filter the time series using your high-pass FIR filter. Plot the M-mode and sonogram again, did it to remove the clutter component?

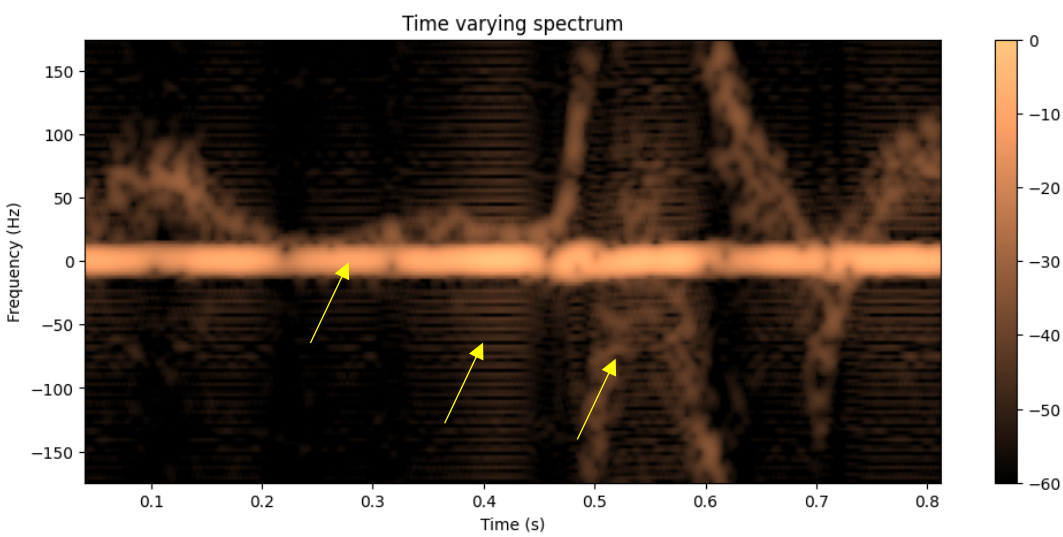
4. Spectrum analysis for *in vivo* data

We will now load an *in vivo* data set and optimize the spectrum estimation in practice. The dataset is from the carotid artery. It contains a 2D data matrix where the first dimension is image depth, and the second dimension is time. *You can select index 50 in the first dimension (depth) for this analysis*.

To read the data in Python:

```
IQ = data['iq']

pars = data['pars']

PRF = pars[0,0]['PRF'].item()

f0 = pars[0,0]['f0'].item()

fs = pars[0,0]['fs'].item()
```

a) Use your spectrum estimator and plot the sonogram, initially using your settings from the lab data above, *but without a clutter filter*. Plot in the log domain (as always), initially using a dynamic range of 60dB. If you normalize the data before plotting, do so to the global maximum in the 2D sonogram (not per slice). The results can look something like the image below depending on settings (NB: I here flipped the image up-down to get the positive velocities to go upwards):



What are we observing in the sonogram at the arrow positions?

b) Optimize your settings as needed to get a nice sonogram for the *in vivo* data. Adjust the window size and degree of overlap, window type, and averaging, to see what happens.

c) Activate the clutter filter from the lab data to attenuate the clutter. Does the filter still work well for in vivo? Adjust the clutter filter settings if needed, i.e. higher stop band and higher cut-off frequency.

d) What is the maximum measurable velocity in the *in vivo* example? Calculate using the correct formula and with the parameters supplied.

   Note: Aliasing is one of the main challenges we deal with in PW-Doppler. Very often the range of measurable velocities is insufficient to capture pathological flow velocities. We have some parameters we can change to influence this range, the transmitted frequency *f0*, and the pulse repetition frequency (*PRF*). The former can be reduced, and the latter can be increased a bit in practice. But still we are often limited and therefore resort to CW-Doppler to measure the very high velocities (e.g. > 2-3 m/s).

e) To better visualize the high blood velocities that are aliased in the spectrum, we can change the baseline of the spectrum, i.e. we can shift the zero-velocity line. We can here do this using the *numpy.roll* function in Python, or the *circshift* function in Matlab. Try to do a baseline shift to set the zero-frequency/velocity line so that you see the full velocity spectrum.

   What is the maximum velocity of the flow?

   What pulse repetition frequency (PRF) would you need to be able to measure this without baseline shift?

   Note: The assumption using the baseline shift is that the flow pattern of interest is unidirectional. If there are several flows in different directions, baseline shift can also cause unwanted wraparound effects. Usually, you know which flow pattern you are interested in and can focus your attention to this. With the baseline shift, you shift the range of measurable velocities from [-vNyquist, vNyquist] to [0, 2*vNyquist].